

Project Abstract

Title: Project T-Rex: The Hardware-Accelerated Chrome Dino

Group Number: 13

1. System Description

The system extends a baseline 3-stage RV32I processor with the RV32M (MUL/DIV/REM) instructions and integrates it with custom memory-mapped I/O peripherals. It solves the hardware-software integration challenge by enabling a bare-metal C program to control physical inputs and real-time outputs through MMIO. High-level functionality is a fully playable "Cacti-Run" (Chrome Dino-style) game with multi-cacti patterns, variable jump physics, live scoring, and optional VGA graphics output and multiplayer integration.

2. Use Cases

1. Real-Time Hardware-Software Interfacing

The system acts as a practical testbed for bridging bare-metal C software with physical FPGA hardware under strict real-time constraints.

- **Practical Applications:** Managing asynchronous user inputs (like hardware button debouncing), controlling state machine transitions, and driving strict display timings (VGA or multiplexed displays).
- **Example Workloads:** The interactive Chrome Dino game loop, which continuously processes collision detection and routes output to memory-mapped peripherals.

2. Hardware-Accelerated Arithmetic

By implementing custom RV32M instructions (MUL, DIV, MAC), the system offloads complex mathematics from the software to dedicated hardware datapaths.

- **Practical Applications:** Executing operations essential for Digital Signal Processing (DSP), embedded machine learning (matrix math), and cryptographic algorithms.

- **Example Workloads:** Calculating dynamic game difficulty, computing cross-round score accumulation, and executing math-heavy C programs like vector dot products and factorials.

3. Algorithmic and Edge-Case Validation

The platform provides a reliable foundation to test standard C algorithms and verify that the custom processor handles mathematical and logical edge cases correctly without pipeline failure.

- **Practical Applications:** Validating format conversions for user interfaces (like binary-to-decimal for the 7-segment display) and ensuring robust hardware-level error handling.
- **Example Workloads:** Running the Euclidean algorithm (GCD) to test remainders, and testing signed vs. unsigned differences, negative numbers, and divide-by-zero exceptions.

3. FPGA Relevance

- Parallel Hardware Peripherals: Unlike software-only polling, the FPGA can run the LFSR random number generator and 7-segment multiplexer concurrently in the background without consuming CPU cycles.
- Deterministic Execution: FPGAs allow for precise, cycle-accurate pipeline stalling and hazard resolution for multi-cycle division operations, which is critical for maintaining a stable real-time game loop.

4. System Scope

Minimum System

- **Pipeline & Core Datapath:** A fully functional 3-stage RISC-V pipeline capable of executing compiled, bare-metal C code (using standard RV32I instructions) without data corruption. This includes a functional RV32M extension (**MUL/DIV**) integrated into the Execute stage with basic cycle-accurate hazard stalling.
- **Basic MMIO & Peripherals:** Implementation of the MMIO Address Decoder to route load/store instructions to the discrete LEDs (for crash states), read from a basic hardware button debouncer, and interface with a hardware 16-bit LFSR (Linear-Feedback Shift Register) for pseudo-random number generation.
- **Simplified Game Loop:** A foundational C program that shifts obstacle data across an array and processes basic jump inputs, proving the software can continuously poll physical FPGA inputs and manipulate output registers.

Goal System (Target Functionality)

- **Advanced Hazard Control:** Robust hardware stall logic that correctly freezes the Fetch/Decode stages during multi-cycle **DIV** operations *without* dropping asynchronous jump inputs from the "sticky" debouncer.
- **Custom Accelerator Integration:** Successful synthesis and software utilization of the custom hardware **MAC** (Multiply-Accumulate) instruction—integrated into the Execute stage and triggered via inline C assembly—to efficiently calculate accumulating scores across the 3 game rounds.
- **Full Hardware-Software Co-Design:** The complete Chrome Dino game loop running seamlessly on the FPGA. This includes the C code actively polling the hardware LFSR for procedural cactus generation, driving the multiplexed 8-digit 7-segment display, and dynamically scaling the game speed utilizing the multi-cycle **DIV** instruction.

Stretch Goals

- **VGA Controller & Tile-Based Rendering:** Designing a custom Verilog VGA controller capable of generating correct horizontal and vertical sync timings for a 640x480 resolution display. The C software will be upgraded to write to a dedicated video memory space (VRAM) mapped via the MMIO, transitioning the visual output from the 7-segment display to a 2D tile-mapped monitor interface.
- **Audio Feedback:** Integrating a hardware buzzer driver (simple square-wave generator via PWM or toggling) that produces a distinct "game over" buzz on the Nexys A7 onboard piezo buzzer. The RISC-V core will trigger the sound via a memory-mapped register write from the C game logic.
- **Multi-Player Extension (Ambitious):** Supporting a two-player mode where two independent Nexys FPGA boards communicate (via simple GPIO wires or UART) to enable head-to-head play on a shared VGA monitor or separate displays, with each board handling its own player inputs and synchronized game state.

5. Design Goals

The system is designed with clearly measurable architectural and hardware performance targets to ensure reliable execution of arithmetic workloads and real-time interactive applications on the FPGA.

- **Latency (cycles):**

The MUL unit should complete multiplication within a single cycle of the Execute stage, while the DIV unit will operate as a multi-cycle functional unit with deterministic latency of approximately 32–34 clock cycles. During multi-cycle execution, the pipeline control logic must correctly stall the Fetch and Decode stages to maintain correct instruction ordering.

- **Throughput:**

Once the pipeline is filled, the processor should sustain a throughput of approximately one instruction per clock cycle for RV32I instructions. Sustain a continuous 8-digit display multiplexing refresh rate above 1kHz to eliminate LED flickering

- **Resource Usage (DSP/BRAM/LUT):**

The design will efficiently utilize FPGA resources by implementing the MUL unit using available DSP slices and storing instruction and data memories in Block RAM (BRAM). Peripheral modules such as MMIO decoders, display controllers, and game support logic will be implemented using LUT-based logic while keeping overall resource utilization within practical limits of the target FPGA board.

- **Correctness and Edge Case Handling:**

All arithmetic instructions must correctly support both signed and unsigned operations (e.g., MUL, DIV, DIVU). Edge cases such as divide-by-zero conditions, negative operand behavior, and arithmetic overflow must produce deterministic and architecturally correct results. Additionally, the MMIO interface must ensure correct synchronization between asynchronous hardware inputs and the processor pipeline so that button inputs are not lost during pipeline stalls.

6. Implementation Approach

We will augment the datapath with an `alu_rv32m` math engine and an `mmio_decoder` to route peripheral data. The control unit (`decoder_ext`, `hazard_unit`) will be updated to handle custom MAC opcodes and multi-cycle stalls.

New Verilog modules: an arithmetic unit in the Execute stage (MUL, DIV, MAC), a hazard unit that generates stall signals, an MMIO address decoder in the memory stage, and four peripheral modules (7-segment multiplexer, 16-bit LFSR, sticky debouncer, LED controller).

Verification proceeds iteratively: module-level Verilog testbenches for the LFSR/debouncer , processor-level simulation for the RV32M instructions using Golden I/O , and software validation by compiling the C game loop with `riscv32-unknown-elf-gcc`.

7. Demonstration Plan

Input: Player presses the physical Nexys A7 button for jump, which is captured by the sticky hardware debouncer and presented via MMIO read register; the 16-bit LFSR runs continuously in hardware for procedural cactus generation.

Processing on FPGA: The bare-metal C game loop reads MMIO inputs, uses MUL/DIV/MAC instructions for physics, difficulty scaling and score accumulation, performs collision detection and obstacle array shifting, then writes updated game state to MMIO output registers.

Output shown: Live track and score rendered on the multiplexed 8-digit 7-segment display with red LEDs indicating crash animation. The **Stretch System** outputs high-resolution sprite graphics to a VGA monitor and sends real-time game state via UART TX for synchronized multiplayer.

8. High Level Timeline

Week	Milestone
23 - 29 Mar	Project concept finalized — game design decided, team roles assigned, system architecture planned, project abstract written and submitted Deliverable: Project Abstract Document
30 Mar - 5 Apr	<code>mul_div_mac_unit.v</code> complete — testbench passes MUL (1-cycle), DIV (32-cycle max), MAC accumulation, and divide-by-zero edge case; <code>hazard_unit.v</code> complete — simulation confirms PC and IF/ID registers freeze correctly for full DIV latency with no instruction corruption Deliverable: Verilog Code + Testbenches for ALU/Hazard Unit + Block Diagram Report
6 Apr - 12 Apr	All four peripherals complete with individual testbenches (<code>seg7_mux.v</code> , <code>lfsr16.v</code> , <code>debouncer.v</code> , <code>led_ctrl.v</code>); <code>mmio_decoder.v</code> routes all addresses correctly in simulation; at least one peripheral verified working on physical FPGA hardware Deliverable: Peripheral Verilog Code + Testbenches + FPGA Demo (Single Peripheral) + Project Design Status Report
13 Apr - 19 Apr	Full game running on physical FPGA — C binary compiled with <code>-march=rv32im</code> , loaded into instruction memory, player can jump, obstacles spawn via LFSR, game speeds up across 3 rounds, score displayed on 7-segment — goal system complete Deliverable: C Source Code + Compiled <code>.hex</code> binary + Full Goal System FPGA Demo + Near-Final System Report
20 - 26 Apr	Stretch goals attempted in priority order (audio → VGA → UART); final hardware testing, edge case hardening, demo rehearsal, report writeup Deliverable: Final Codebase + Final Report + 4-Minute Demo Video + Final FPGA Demo

9. Team Work Plan

- Responsibilities per member
- Collaboration approach

Member	Responsibilities
Ananya Bhamidipaty	Peripherals & MMIO: Develops peripheral modules including seg7_mux.v, debouncer.v, led_ctrl.v, and lfsr16.v, along with the mmio_decoder.v for address routing. Ensures accurate memory-mapped I/O behavior and proper address decoding. Writes and validates testbenches for all peripherals.
Surbhi Kumari	FPGA & Hardware Integration: Manages the complete FPGA setup and bring-up on the Nexys board. Responsibilities include authoring the physical constraints file (.xdc), performing precise pin mapping, and overseeing bitstream generation. Leads hardware-level testing of peripherals and supports full system deployment and debugging.
Avni Jhalani	Hazard & Pipeline Control: Responsible for implementing the hazard_unit.v, ensures correct PC freeze logic and IF/ID stall handling during long-latency DIV operations. Verifies that no instruction corruption occurs during stalls and assists in pipeline-level debugging during the integration phase.
T S S Sreeja	ALU & Arithmetic Unit: Responsible for designing and implementing mul_div_mac_unit.v, including single-cycle multiplication, iterative division (≤ 32 cycles), and MAC accumulation. Handles edge cases such as divide-by-zero and ensures correctness through a comprehensive testbench. Also supports debugging of arithmetic-related issues during system integration.
Sanjeebani Parida	System Integration, C Code & Verification: Develops the C-based game logic, including player control, obstacle generation using LFSR, and scoring. Compiles the program with -march=rv32im to generate the .hex file for execution. Integrates all modules and performs end-to-end testing and debugging of the complete system.

10. Github Link

- Provide link to private repo - <https://github.com/sanjeebani14/CS224-Project>
-